Tournament Auction Platform

Database Review and Final Blueprint (your simplified design vs. the earlier 27-collection design)

Stack: React + TypeScript, Node.js/Express + TypeScript, Mongoose, MongoDB Atlas (free tier), Socket.IO. Priorities: fast live auction (~100 viewers), small storage (~128 MB), simplicity.

1. Verdict

Your simplified design is a good foundation and is close to sufficient. The earlier 27-collection design is over-engineered for a portfolio project: about half of its collections (sessions, ledger, audit log, CSV imports, lots, sales, qualification decisions, memberships, form builder, settings) solve problems you do not have. Do not adopt it wholesale.

Your design needs **a small number of real corrections** (listed below, each with a concrete failure scenario), and I recommend merging three collections into others. Result: **13 collections** (down from your 15 and the old 27), with the hot auction path touching exactly one small document.

- **Essential fixes (must do before coding):** E1 to E10 in Section 2.
- **Recommended simplifications:** embed Group inside Stage, embed Innings inside Fixture, move current-bid state to the Auction document, move bid history to a lean bids collection.
- **Can postpone:** notifications, player_stats (both are additive, nothing depends on them).
- **Not needed at all (for now):** everything in Section 8.

2. What must change in your design, and why

Legend: **E** = essential (a real bug or integrity risk), **R** = recommended simplification, **O** = optional. Each item has a concrete scenario.

#	Your design	Change	Why / example scenario
E1	Player.status is an "object"	Make it a plain string enum: pending | sold | unsold | withdrawn.	An object cannot be indexed or filtered cleanly. Query "all unsold players" becomes status.value with no clear schema, and Mongoose will happily store {sold:true, unsold:true} together.
E2	Player.isSold	Remove. status already says it.	Two fields for one fact drift apart: a failed undo leaves status = pending but isSold = true, so the player vanishes from the pool but also shows in no squad.
E3	Player.allBids array (embedded)	Remove. Keep the live bid on the Auction doc; history in a small bids collection (Section 3).	Every Player list (pool, squads, public page) would drag the whole bid array along; and the sale/undo logic would update a big, frequently changing document that is also read by 100 viewers.
E4	User.secureCode	Store as secureCodeHash (bcrypt), select:false. Add failedAttempts + lockedUntil.	If the code is plain text, a DB leak lets anyone reset or delete accounts. A short code with no attempt limit can be brute-forced: 10,000 guesses for a 4-digit code.
E5	Tournament has no points rules	Add pointsConfig {win, tie, noResult, loss}.	Your Standings has "points" but nothing says how many a win gives. Hard-coding 2 breaks the moment an organizer wants 3 for a win or 1 for a tie.
E6	Fixture.result type completed | no_result, plus status no_result	status: scheduled | completed | cancelled. result.type: win | tie | no_result. winnerId nullable. Drop loserId.	(a) A tied match has no winner: your result cannot express it, yet Standings has to award tie points. (b) Status "no_result" and result.type "no_result" are two copies; after a correction they can disagree. (c) loserId is derivable from teamA/teamB and winnerId.
E7	Standings: no "tied", no source of truth	Add tied. Treat Standings as fully rebuildable from fixtures (rebuild function, never incremental +/-).	Correcting match 12 from "A won" to "tie" with incremental updates means: subtract old result, add new result. If the request is retried after a timeout you subtract twice and the table is permanently wrong. A rebuild from fixtures is idempotent.
E8	Stage.numGroups	Remove (it is groups.length).	numGroups = 4 but only 3 group documents exist after the organizer deletes one. Which is true?
E9	Tournament.requestId AND TournamentRequest.tournament_id	Keep ONE direction, and make it unique: tournaments.requestId (unique).	Owner double-clicks "Approve": two tournaments are created from one request. A unique index on tournaments.requestId makes the second insert fail safely. The request page can look up its tournament through this index.
E10	Registration.accountType = star | gem | star verified | gem verified	Split into accountType (star | gem) + isVerified (Boolean).	"Show all verified players" needs $in on two strings today and breaks when a third type appears. Two independent facts should be two fields.
R1	Group as separate collection	Embed as stage.groups [{_id, name, order, teamIds, qualificationCount}].	Moving a team from Group A to Group B is two writes on two documents: after a crash the team is in both groups (or neither) and fixtures generate wrongly. In one Stage document it is one atomic write. Groups are at most ~8 small objects.

#	Your design	Change	Why / example scenario
R2	Innings as separate collection	Embed as fixture.innings (array of 1-4).	An innings is about 1.5 to 2 KB; a whole fixture is about 4 KB, far below 16 MB. Correcting a score is ONE write for result + scorecard instead of two or three. Scorecard page = one read. Saves a collection and 3 indexes. Use a projection (exclude innings) for fixture lists.
R3	Auction.currentRound ("39th player")	Rename currentOrder; add Player.auctionOrder; add live-state fields to Auction.	Without a stored order, "next player" after an undo or a withdrawn player is ambiguous. With auctionOrder, next = smallest order among pending players.
R4	Standings.type nrr | custom	Replace by override {points, nrr, position} (nullable).	"custom" only says that someone edited something. The override says exactly which values and keeps system values visible, so a rebuild does not wipe the organizer's manual ranking.
R5	Fixture playoff teams unknown	teamAId / teamBId nullable + optional teamALabel / teamBLabel ("Winner of SF1").	Semi-final is created before the league ends; teams are not known yet, but the bracket must display.
R6	Tournament.status auction_* values duplicate Auction.status	Tournament.status uses a single "auction" phase; detail lives in auction.status.	Auction paused, tournament still says auction_in_progress, public page says "LIVE". One owner for each fact.
R7	Registration.Type "Player / Captain"	Rename wantsCaptain (Boolean).	A registrant is always a player; "captain" is only a volunteer preference. The captain is later assigned on Team.captainUserId.
O1	TournamentRequest.sendAt	Drop; createdAt is the same value.	Two timestamps for one event.
O2	Registration "draft" status	Keep only if you want server-side drafts.	Drafts cost documents and an extra state. A frontend-only draft is simpler. Your call.

What your design already handles correctly (keep as is)

- **User / TournamentRequest / Tournament split:** approval flow with status + response is right. No tournament_members needed: organizer = tournaments.organizerId, captain = teams.captainUserId. Global role stays only user | owner.
- **Team with purse {initial, current} and squadCount:** good summary data. Keep; do not replace by a ledger.
- **Registration separate from Player:** keep (reason: private contact numbers and rejected/draft rows must never leak into the public auction pool).
- **Stage / Fixture / Standings / Player statistics:** structure is right; only the corrections above.
- **Auction document with bidIncrement rules and timer:** keep; add live-state fields.
- **Notifications:** fine as designed; optional.

3. Auction performance (highest priority)

3.1 Honest evaluation of embedded allBids in Player

A bid is about 50 bytes in BSON, so 1,000 bids is only ~50 KB: **the 16 MB limit is not your problem**, and I would not reject embedding on size alone. The real problems are different:

- **Hot-document contention:** in a busy auction every bid writes the same Player document, and you also need to compare with the current price. That is fine for correctness (single-document updates are atomic) but you would still need a second place (team purse check) so embedding saves nothing on the critical path.
- **Read amplification:** the Player document is read everywhere (pool list, squads, public page, stats joins). With embedded bids each of those reads loads the history unless you remember to project it out every single time. One forgotten projection and 100 viewers download 100 players x 30 bids.
- **Unbounded growth:** bids only ever grow; a document that grows is relocated/rewritten more often. Not dangerous at 1,000, but it is a design smell for a field you rarely read.
- **Undo:** embedded undo is easy ($pop), which is the one real advantage. The separate collection costs one extra query (the last two bids) and is still simple.

3.2 Recommendation

Option	Verdict
Embedded allBids in Player	Works, but not recommended: read amplification and mixing "live state" with "history".
Separate lean bids collection (accepted bids only: playerId, teamId, amount, createdAt)	**Recommended.** About 90 bytes per bid, one index. Written AFTER the live state is updated, so it is not on the speed-critical path. Used only for history display and undo.

Option	Verdict
Separate auction_lots collection	**Not needed.** One auction turn per approved player means a lot is just "the current player". Player holds the result (status, teamId, soldPrice); Auction holds the live state (currentPlayerId, currentAmount, currentTeamId, timerEndsAt). A lot collection only pays off for re-auction rounds with per-attempt history; if you later want unsold players re-offered, set them back to pending (their bids from the earlier attempt stay in bids).
Full old design (lots + bids + sales + ledger)	Not needed. Four extra collections do not make a bid faster; a bid is one indexed atomic update either way.

3.3 The bid path (one atomic update, no transaction)

Validation that needs no database write happens first in Node memory (cache team purses/squad counts at auction start and refresh them after each sale). The single source of truth for "who is highest" is the Auction document, updated by **one conditional findOneAndUpdate**. MongoDB serialises writes on one document, so two simultaneous bids can never both win.

```
1. Auth: socket user is captain of team (team.captainUserId == userId).   [memory/DB cache]
2. Rules (in code):
   - auction.status == 'live' and auction.currentPlayerId == playerId
   - teamId != auction.currentTeamId                    (cannot outbid yourself)
   - amount >= (currentAmount == 0 ? basePrice : currentAmount + stepFor(currentAmount))
   - team.squadCount < tournament.teamConfig.maxPlayers
   - amount <= team.purse.current - (minPlayers - squadCount - 1) * basePrice
3. Atomic write (this is the real guard):
   const a = await Auction.findOneAndUpdate(
     { _id: auctionId, status: 'live', currentPlayerId: playerId,
       currentAmount: { $lt: amount } },
     { $set: { currentAmount: amount, currentTeamId: teamId,
               timerEndsAt: new Date(Date.now() + timerSeconds * 1000) } },
     { new: true });
   if (!a) -> reject 'OUTBID_OR_CLOSED' to that captain only.
4. io.to(`auction:${tournamentId}`).emit('bid', { playerId, teamId, amount, timerEndsAt });
5. Bid.create({ playerId, teamId, amount })   // history, after broadcast, not awaited by the hot path
```

Race example: price is 100, step is 5. Captains X and Y both send 105 at the same millisecond. Both pass step 2 (they read 100). The first write matches currentAmount < 105 and wins. The second finds currentAmount = 105, the filter fails, and Y receives "outbid". Nobody is double-charged because nothing is charged until the sale.

3.4 Broadcast with Socket.IO

- One room per tournament: socket.join("auction:" + tournamentId). Viewers and captains are in the same room; only captains may emit "placeBid" (checked on the server, never trust the client).
- Send small deltas (playerId, teamId, amount, timerEndsAt, server time), not full documents. 100 viewers x ~100 bytes per bid is negligible.
- On connect/reconnect the client calls one REST/ack "getAuctionState" (auction doc + current player + team summaries). This repairs any missed message.
- Timer: keep a setTimeout per live auction in server memory; ALSO store timerEndsAt in the DB. After a server restart, read live auctions and recreate the timers. Clients display countdown from timerEndsAt minus server-time offset.
- Free hosting note: run a single Node instance. Multiple instances would need the Socket.IO Redis adapter, which you do not need.

3.5 Sale (organizer clicks Sold, or timer expires)

Sale touches three documents (auction, team, player), so use one MongoDB transaction here only. Atlas free tier runs a replica set, so transactions work. Order matters: update the Auction document first, using the exact amount and team you read as the filter, so a bid that sneaks in aborts the sale.

```
session.withTransaction(async () => {
  // 1. close the lot only if nothing changed since we read it
  a = Auction.updateOne({ _id, currentPlayerId: pid, currentAmount: amt, currentTeamId: tid },
        { $set: { currentPlayerId: null, currentAmount: 0, currentTeamId: null,
                  timerEndsAt: null } }, { session });          // must match 1 doc
  // 2. guarded purse debit (cannot go below 0, cannot exceed max squad)
  t = Team.updateOne({ _id: tid, 'purse.current': { $gte: amt },
                       squadCount: { $lt: maxPlayers } },
        { $inc: { 'purse.current': -amt, squadCount: 1 } }, { session });   // must match 1
  // 3. mark player (only if still pending)
  p = Player.updateOne({ _id: pid, status: 'pending' },
        { $set: { status: 'sold', teamId: tid, soldPrice: amt, soldAt: new Date() } },
        { session });                                                // must match 1
}); // any "matched 0" -> throw -> whole transaction rolls back
then: emit 'sold'; if amt == 0 (no bids): player.status = 'unsold' instead, no team update.
```

3.6 Undo operations

- **Undo last bid** (single document logic, no transaction): find the two highest bids for the current player (sort amount desc, limit 2). Atomically revert the Auction with filter {currentPlayerId, currentAmount: top.amount, currentTeamId: top.teamId} to the second bid (or 0/null if none) and reset timerEndsAt; then delete the top bid. Because amounts only increase, the filter prevents undoing a bid that was already outbid.
- **Undo sale** (transaction): player -> pending (clear teamId, soldPrice, soldAt); team purse.current += soldPrice, squadCount -= 1; allowed only while no player is currently on the block. Re-queue by giving it the next auctionOrder or restoring its old order. Because the purse is refunded in the same transaction as the player change, they can never disagree.
- **Purse verification (instead of a ledger):** purse.current must equal purse.initial minus the sum of soldPrice of that team's sold players. Write a tiny recompute script/endpoint; you get the audit benefit without a ledger collection.

3.6.1 Why the Team stores purse.current

Reading and guarding a number is faster and safer than aggregating players on every bid, and it is a tiny summary (as you said: do not remove useful summary data). It is rebuildable from players, which is what makes a ledger unnecessary at this scale.

4. Storage estimate (small tournament)

Assumptions: 16 teams, 150 registered users, 120 registrations, 100 players, 60 fixtures with scorecards, 1,500 accepted bids, 300 notifications. BSON sizes are rough; figures are for planning, not exact.

Collection	Docs	Approx. size each	Total
users (shared by all tournaments)	150	250 B	~40 KB
registrations	120	450 B	~55 KB
players	100	250 B	~25 KB
teams	16	300 B	~5 KB
fixtures (innings embedded)	60	~4 KB	~240 KB
standings + player_stats	16 + 100	150 B / 250 B	~30 KB
bids	1,500	~90 B	~135 KB
notifications (TTL cleaned)	300	200 B	~60 KB
stages, auction, tournament, requests	~10	1 to 2 KB	~15 KB
Documents total			**~0.6 MB**
Indexes (about 20 indexes, small collections)			**~0.3 to 0.6 MB**

Total about 1 to 1.5 MB per small tournament, so 128 MB holds many tournaments. Storage is not your constraint; **unnecessary collections and indexes are the only real waste**. The old design adds ~60 indexes, a purse ledger, an audit log, rawContent of CSVs and nameHistory arrays, which would multiply this several times for no benefit.

Store vs calculate

Store	Calculate when needed
team.purse.current, team.squadCount (hot, guarded counters)	Strike rate, economy, average, batting average (from stored runs/balls)
standings rows (public page speed; rebuilt from fixtures)	Number of groups, number of bids on a player, loserId of a match
player_stats rows (leaderboards; rebuilt from fixtures)	Remaining purse check by recompute (verification script only)
player.displayName, accountType, isVerified (snapshot, shown in the auction without a join)	Number of players in a team (use squadCount) / unsold count (query)

5. Data consistency: relationship by relationship

Relationship	Possible inconsistency	Simplest reliable prevention
User -> Request -> Tournament	Approval creates a tournament and flips the request. A crash between the two leaves an approved-looking tournament with a pending request.	Do both in one transaction. Unique index tournaments.requestId makes a repeat approval harmless. Organizer rights come from tournaments.organizerId, so no role flag on User can go stale.
Registration -> Player	Organizer approves; the player insert succeeds but the registration update fails (or reverse).	Insert Player first (unique registrationId = idempotent), then set registration accepted. Retrying is safe. Block withdrawing/rejecting once the player is sold.
Team / Player / Auction	Sold player but team not charged, or team charged twice.	Sale and undo-sale are the ONLY transactions (Section 3.5). Guarded $inc with purse.current $gte price. Recompute script to verify.

Relationship	Possible inconsistency	Simplest reliable prevention
Stage -> Group -> Fixture -> Standings	Team moved between groups after fixtures exist; deleting a stage leaves orphans.	Groups embedded (atomic moves). Lock team allocation once any fixture exists for the stage. Unique {tournamentId, matchNumber}. Standings unique {stageId, teamId}. Delete a stage with its fixtures/standings in one service function.
Fixture -> Innings -> Player stats	Corrected score leaves standings/leaderboards stale or double-counted.	Update result + innings in ONE fixture write. Then call rebuildStandings(stageId) and rebuildPlayerStats(tournamentId), both recompute from completed fixtures with upserts. Revert a result = set status scheduled, clear result and innings, rebuild.
Corrected results	Match changes after the group stage already produced playoff teams.	Block corrections on a stage once a later stage has started, OR allow and show a warning; qualification is just read from standings, so nothing is stored that can drift.
Auction -> Notifications	A notification per bid would explode writes.	Live bids go over Socket.IO only. Notifications for durable events only: request approved/rejected, registration accepted/rejected, captain assigned, auction scheduled.

6. MongoDB indexes: the minimum useful set

Rule used: an index is kept only if it enforces uniqueness or serves a query that runs often on a collection that can grow. For collections with at most a few hundred documents per tournament, the _id or a unique-index prefix is enough, and the rest is scanned in memory.

Collection	Index	Purpose
users	unique {email}	login, no duplicate accounts (store email lowercased)
users	unique {username} with collation strength 2	case-insensitive unique username without a duplicate "usernameLower" field
users	unique partial {role} where role = "owner"	guarantees only one owner can ever exist
tournament_requests	{status, createdAt:-1}	owner dashboard: pending requests, newest first
tournament_requests	unique partial {userId} where status = "pending"	one open request per user; also serves "my request"
tournaments	unique {slug}	pretty URLs
tournaments	unique {requestId}	idempotent approval (E9)
tournaments	{organizerId}	"my tournaments"
teams	unique {tournamentId, name} (collation strength 2)	no duplicate team names; its prefix also lists a tournament's teams
teams	unique partial {tournamentId, captainUserId} where captainUserId exists	a user captains at most one team per tournament; "my team" lookup
registrations	unique {tournamentId, userId}	one submission per user; prefix lists a tournament's registrations
players	unique {registrationId}	idempotent approval (one player per registration)
players	{tournamentId, auctionOrder}	pool list and "next player" in auction order
players	partial {teamId} where teamId exists	team squad
stages	unique {tournamentId, position}	ordered stages, no duplicate position
fixtures	unique {tournamentId, matchNumber}	no duplicate match numbers
fixtures	{stageId}	standings rebuild and stage pages
standings	unique {stageId, teamId}	one row per team per stage; table query by stageId
player_stats	unique {tournamentId, playerId}	one row per player; leaderboards sort the few hundred rows in memory, so no extra sort indexes
auctions	unique {tournamentId}	one auction per tournament
bids	{playerId, amount:-1}	last/highest bids for undo and history
notifications	{recipientId, createdAt:-1}	inbox and unread count
notifications	TTL {createdAt}, expireAfterSeconds 5184000 (60 days)	automatic cleanup to save storage

Total: 22 indexes beyond _id, nearly all on tiny collections. Do not add indexes on status fields until a real query is slow.

7. Verdict summary

Question	Answer
Is my simplified schema sufficient?	Yes, after fixes E1 to E10. The structure and scope are right.
Fields to remove	Player.isSold, Player.allBids, Stage.numGroups, Request.sendAt, Fixture.result.loserId, Fixture status no_result, Registration accountType combos, Tournament.requestId OR Request.tournament_id (keep one).

Question	Answer
Fields to add	User.secureCodeHash/failedAttempts/lockedUntil/tokenVersion/deletedAt, Tournament.pointsConfig, Standings.tied + override, Fixture.result.type tie, Player.auctionOrder/soldAt, Auction live-state fields, Registration.isVerified.
Fields to rename	secureCode -> secureCodeHash, Type -> wantsCaptain, currentRound -> currentOrder, timeline (Auction) -> schedule, tournament_timeline -> timeline.
Collections to keep	users, tournament_requests, tournaments, teams, registrations, players, stages, fixtures, standings, auctions, bids (+ notifications, player_stats).
Collections merged	Group -> inside stages; Innings -> inside fixtures.
Postpone	notifications, player_stats (build after the auction and fixtures work).
Serious design flaws to fix first	E3 (bids), E4 (secure code), E6/E7 (results and standings correctness), E9 (double approval).

8. Collections from the earlier design that you do NOT need

Old collection	Why it existed	Why you can skip it now
system_settings	One-time owner bootstrap	Create the owner with a seed script/env var once; the partial unique index guards the single owner.
sessions, account_tokens	Refresh-token rotation, hashed reset tokens	Use a JWT + tokenVersion on User. Password reset with username + email + secure code needs no token table.
account_deletion_requests	Two-step deletion workflow	Do two confirmation calls in the API; soft-delete (deletedAt, anonymize) the user.
tournament_members	Per-tournament roles	organizer = tournaments.organizerId, captain = teams.captainUserId. One organizer is enough for v1.
registration_forms	Organizer-built form schema	You have a fixed registration form; a form builder is a future enhancement.
qualification_decisions	Stored qualification outcomes	Qualifiers are read from standings (qualificationCount). Stored decisions can drift from standings.
csv_imports	CSV upload history	Parse CSV in the request and write innings directly (source: csv). Keep the file out of the DB.
auction_lots	Per-round lot state	One turn per player: state lives in Auction + Player (Section 3).
auction_sales	Sale records with reversals	Player.status/teamId/soldPrice/soldAt + undo transaction replace it.
purse_transactions	Append-only purse ledger	Purse is recomputable from sold players; verification script instead.
audit_logs	History of every edit	Not required. Add later only if the owner must see who changed a result.

Future enhancements (explicitly not for v1): co-organizers, form builder, re-auction rounds with history, audit log, purse ledger, Redis adapter for multi-instance Socket.IO.

9. Final consolidated database design (implementation blueprint)

Conventions: all collections use Mongoose **timestamps: true** (createdAt, updatedAt) unless noted. Money is an integer in virtual credits. All ObjectId references use ref. Validation shown is the minimum to enforce in the Mongoose schema and in request validation (e.g. zod).

1. users — Accounts: normal users and the single owner

Field	Type	Req	Rules / notes
username	String	Y	3-30 chars, letters/digits/underscore, original casing kept
email	String	Y	lowercase, trimmed, valid format
passwordHash	String	Y	bcrypt (cost 10-12); select:false; never returned by API
secureCodeHash	String	Y	bcrypt hash of user-chosen code (min 6 chars); select:false
role	Enum	Y	user | owner; default user; signup can only create user
tokenVersion	Number	Y	default 0; increment on password reset/deletion so old JWTs stop working
failedAttempts	Number	Y	default 0; counts wrong password OR secure code attempts; reset on success
lockedUntil	Date	N	set 15 min ahead after 5 failures
deletedAt	Date	N	soft delete: on deletion set username "deleted_<id>", email "<id>@deleted.invalid", clear hashes; keep _id so history stays valid

Indexes: unique {email}; unique {username} collation {locale:"en", strength:2}; unique partial {role} where role = "owner"

Notes: Queries on username must use the same collation to use the index. Block deletion of a user who organizes an active tournament.

2. tournament_requests — User asks the owner for permission to run a tournament

Field	Type	Req	Rules / notes
userId	ObjectId(users)	Y	requester
name	String	Y	proposed tournament name, 3-120 chars
description	String	N	max 2000 chars
status	Enum	Y	pending | accepted | rejected | withdrawn; default pending
response	String	N	owner message, required when rejected
reviewedAt	Date	N	set when status leaves pending

Indexes: {status, createdAt:-1}; unique partial {userId} where status = "pending"

Notes: createdAt replaces sendAt. The tournament is found through tournaments.requestId.

3. tournaments — Central document of a tournament

Field	Type	Req	Rules / notes
name	String	Y	3-120 chars
slug	String	Y	url-safe, unique, generated from name
description	String	N	max 5000
organizerId	ObjectId(users)	Y	the organizer (request requester)
requestId	ObjectId(tournament_requests)	Y	unique; guards double approval
visibility	Enum	Y	public | private; default public
status	Enum	Y	draft | registration_open | registration_closed | auction | tournament_in_progress | completed | cancelled
timeline	Object	N	{registrationStart, registrationEnd, auctionStart, auctionEnd, tournamentStart} all Date; registrationEnd after registrationStart
contact	Object	Y	{name: String, phone: String}
teamConfig	Object	Y	{totalTeams (2-64), minPlayers, maxPlayers}; minPlayers at most maxPlayers
pointsConfig	Object	Y	{win:2, tie:1, noResult:1, loss:0} integers, editable; changing it triggers rebuildStandings
currentStageId	ObjectId(stages)	N	shown as "current stage" on the public page

Indexes: unique {slug}; unique {requestId}; {organizerId}

Notes: Auction detail (live, paused...) is in auctions.status; this status only says that the tournament is in its auction phase.

4. teams — Team with captain and purse summary

Field	Type	Req	Rules / notes
tournamentId	ObjectId	Y	parent

Field	Type	Req	Rules / notes
name	String	Y	2-40 chars, unique per tournament (case-insensitive)
shortCode	String	Y	2-5 uppercase chars
captainUserId	ObjectId(users)	N	null until assigned; user must have an accepted registration
purse	Object	Y	{initial: Number, current: Number}; current at least 0 (guard in every update)
squadCount	Number	Y	default 0; incremented/decremented only inside sale/undo transactions

Indexes: unique {tournamentId, name} collation strength 2; unique partial {tournamentId, captainUserId} where captainUserId is an ObjectId

Notes: purse.initial is copied from auction.purse.perTeam when teams are created or the auction is configured.

5. registrations Player submissions (private data lives here)

Field	Type	Req	Rules / notes
tournamentId	ObjectId	Y	parent
userId	ObjectId(users)	Y	registrant
actualName	String	Y	real name; private
displayName	String	Y	name shown publicly, 2-40 chars
accountType	Enum	Y	star | gem
isVerified	Boolean	Y	default false (set by organizer)
contact	Object	Y	{number: String, country: String}; private
wantsCaptain	Boolean	Y	default false
status	Enum	Y	pending | accepted | rejected (add draft only if you want server drafts)
response	String	N	organizer feedback; required when rejected; user edits and resubmits (status back to pending)
submittedAt	Date	N	set when submitted

Indexes: unique {tournamentId, userId}

Notes: Never expose actualName/contact in public or auction APIs. Contact only for organizer.

6. players Approved registration = auction pool entry (lean, public)

Field	Type	Req	Rules / notes
tournamentId	ObjectId	Y	parent
userId	ObjectId(users)	Y	copied from registration
registrationId	ObjectId(registrations)	Y	unique source
displayName	String	Y	snapshot at approval (avoids a join in the auction screen)
accountType	Enum	Y	star | gem; snapshot
isVerified	Boolean	Y	snapshot
status	Enum	Y	pending | sold | unsold | withdrawn; default pending
teamId	ObjectId(teams)	N	set only when sold
soldPrice	Number	N	integer at least 0; set only when sold
soldAt	Date	N	used to find the most recent sale for undo
auctionOrder	Number	N	1..N, assigned when the auction starts (random or manual)

Indexes: unique {registrationId}; {tournamentId, auctionOrder}; partial {teamId} where teamId exists

Notes: Invariant: status = sold if and only if teamId, soldPrice and soldAt are set. isSold and bids are intentionally NOT stored here.

7. stages One stage of a tournament, with embedded groups

Field	Type	Req	Rules / notes
tournamentId	ObjectId	Y	parent
name	String	Y	organizer-defined
position	Number	Y	1..N, unique per tournament
type	Enum	Y	groups | playoff | custom
status	Enum	Y	ready | in_progress | completed; default ready
teamIds	[ObjectId(teams)]	Y	participants of the stage
fixtureMode	Enum	Y	auto | manual
allocationMethod	Enum	N	auto | manual (groups stages only)

Field	Type	Req	Rules / notes
groups	[Object]	N	[{_id (auto), name, order, teamIds[], qualificationCount}] max about 8 groups; a team appears in at most one group of the stage

Indexes: unique {tournamentId, position}

Notes: numGroups is not stored (groups.length). Group _id is what fixtures.groupId and standings.groupId refer to.

8. fixtures Matches with result and embedded scorecard

Field	Type	Req	Rules / notes
tournamentId	ObjectId	Y	parent
stageId	ObjectId(stages)	Y	parent stage
groupId	ObjectId	N	embedded group _id; null for playoff/custom
matchNumber	Number	Y	unique per tournament
roundLabel	String	N	"Match 46", "Semi Final 2"
teamAId, teamBId	ObjectId(teams)	N	nullable for undecided playoff slots; must differ
teamALabel, teamBLabel	String	N	display text while team is unknown ("Winner of SF1")
description	String	N	venue and notes, max 500
status	Enum	Y	scheduled | completed | cancelled; default scheduled
result	Object	N	{winnerId (null for tie/no_result), type: win | tie | no_result, summary: String}; present only when completed
scheduledAt, completedAt	Date	N	completedAt set when result is saved
innings	[Object]	N	max 4 (super over). Each: {inningsNo, battingTeamId, bowlingTeamId, source: manual|csv, totals:{runs, wickets, balls}, batting:[{playerId, runs, balls, fours, sixes, isOut}], bowling:[{playerId, balls, runs, maidens, wickets}], fallOfWickets:[{runs, wickets, over}]}

Indexes: unique {tournamentId, matchNumber}; {stageId}

Notes: Validation: winnerId must be teamAId or teamBId; batting and bowling teams must be the two fixture teams and differ; playerIds must belong to those teams. Use projection {innings:0} for fixture lists. After any write to result/innings call rebuildStandings and rebuildPlayerStats.

9. standings Points table row; fully derived from fixtures

Field	Type	Req	Rules / notes
tournamentId	ObjectId	Y	parent (cleanup)
stageId	ObjectId	Y	stage
groupId	ObjectId	N	embedded group _id; null for league/playoff
teamId	ObjectId(teams)	Y	team
played, won, lost, tied, noResult	Number	Y	default 0
points	Number	Y	computed from tournament.pointsConfig
nrr	Number	Y	computed; (runsFor/ballsFaced - runsAgainst/ballsBowled) x 6 style formula, from fixture innings totals
position	Number	Y	computed rank after applying override
override	Object	N	{points, nrr, position, reason} organizer manual values; a rebuild keeps it and applies it

Indexes: unique {stageId, teamId}

Notes: rebuildStandings(stageId): read completed fixtures of the stage, recompute every row, bulkWrite upserts. Idempotent; safe to retry. Must be called after any result change or pointsConfig change.

10. player_stats Per-player tournament aggregates; derived (can be postponed)

Field	Type	Req	Rules / notes
tournamentId	ObjectId	Y	parent
playerId	ObjectId(players)	Y	subject
batting	Object	Y	{matches, innings, runs, balls, fours, sixes, highScore, notOuts}
bowling	Object	Y	{matches, balls, runs, wickets, bestWickets, bestRuns}

Indexes: unique {tournamentId, playerId}

Notes: Strike rate, economy, averages are calculated on read. Leaderboards: query by tournamentId and sort in memory (a few hundred rows). rebuildPlayerStats(tournamentId) aggregates fixture innings.

11. auctions Auction configuration + the single live state

Field	Type	Req	Rules / notes
tournamentId	ObjectId	Y	unique; one auction per tournament
status	Enum	Y	draft | scheduled | live | paused | completed
schedule	[Object]	N	[{startsAt: Date, endsAt: Date}] auction sessions
purse	Object	Y	{perTeam, basePrice}; integers, basePrice at most perTeam
timerSeconds	Number	Y	countdown after the player opens and after each bid (e.g. 15-60)
bidIncrements	[Object]	Y	[{from, step}] sorted ascending by from, first from equals basePrice, e.g. [{1000,1000},{50000,5000},{100000,10000}]
playerOrder	Enum	Y	auto | manual
currentPlayerId	ObjectId(players)	N	null when nobody is on the block
currentOrder	Number	N	auctionOrder of the current player ("39th player")
currentAmount	Number	Y	default 0 (no bid yet)
currentTeamId	ObjectId(teams)	N	highest bidder
timerEndsAt	Date	N	server-controlled; persisted so timers survive a restart

Indexes: unique {tournamentId}

Notes: This is the ONLY document written on the bid hot path. Every live write uses a conditional filter (Section 3).

12. bids Accepted bid history (not on the hot path)

Field	Type	Req	Rules / notes
playerId	ObjectId(players)	Y	player the bid is for
teamId	ObjectId(teams)	Y	bidding team
amount	Number	Y	integer; strictly increasing per player
createdAt	Date	Y	server time; no updatedAt

Indexes: {playerId, amount:-1}

Notes: timestamps: {createdAt: true, updatedAt: false}. Rejected bids are not stored. Undo deletes the top bid.

13. notifications In-app notifications for durable events only (can be postponed)

Field	Type	Req	Rules / notes
recipientId	ObjectId(users)	Y	target user
type	String	Y	tournament_request | request_response | registration_response | captain_assigned | auction_scheduled
body	String	Y	short text, max 300
link	Object	N	{kind, id} deep link
readAt	Date	N	null = unread

Indexes: {recipientId, createdAt:-1}; TTL {createdAt} expireAfterSeconds 5184000

Notes: Never write notifications for individual bids; use Socket.IO.

10. Suggested implementation order

- 1) users, auth, owner seed, tournament_requests, tournaments.
- 2) teams, registrations, players (approval flow with unique indexes).
- 3) auctions + bids + Socket.IO bid path, sale and undo (build tests with two simulated captains bidding simultaneously).
- 4) stages (with groups), fixtures (with innings), rebuildStandings, then player_stats and notifications.